The Prompt Ladder: Assignment

Iterative Prompt Optimization for 3D Medical Image Processing (PyTorch & MONAI)

Prompt 0:

Help me train a pyTorch model for medical imaging

Here is your **Baseline Output (Run 0)** based on your prompt: *"Help me train a pyTorch model for medical imaging"*.

Output (Baseline / Run 0)

Training a PyTorch model for medical imaging involves a standard machine learning workflow with a few specialized considerations for medical datasets. Here is a high-level overview and starter template:

1. Standard PyTorch Training Script

```python
import torch
import torch.nn as nn
```

```
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torchvision.transforms as Transforms

# Dummy Dataset Placeholder
class MedicalDataset(Dataset):
    def __init__(self):
        # Placeholder for DICOM / NIfTI or PNG loading
        pass

    def __len__(self):
        return 100

    def __getitem__(self, idx):
        # Returning dummy image (3 channels, 224x224) and
label
        return torch.randn(3, 224, 224), torch.tensor(0)

# 1. Setup Model, Loss, Optimizer
device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
model = torchvision.models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, 2)  # Binary
classification
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)

# 2. Data Loaders
```

```
train_loader = DataLoader(MedicalDataset(), batch_size=16,
shuffle=True)

# 3. Training Loop
num_epochs = 5
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch [{epoch+1}/{num_epochs}], Loss:
{running_loss/len(train_loader):.4f}")
```

Baseline Analysis & Next Step

- **What Changed in Prompt:** None (Baseline).
- **What Actually Improved in Output:** N/A (Initial run).
- **What Still Failed:** The code relies on **torch.randn** dummy data, assumes generic 2D RGB classification, ignores specialized modalities (e.g., 3D MRI/CT vs 2D X-ray, DICOM headers), lacks evaluation metrics suitable

for imbalanced medical data (like Dice score or AUC-ROC), and offers no specific architecture guidance.

- **What to Try Next:** Pick **one** single layer from your allowed list (Clearer Goal, Defined Audience, Real Context, Specified Output Format, Constraints, Examples, Quality Criteria, Review Instructions, Stated Assumptions, or Verification Requirements) to refine the prompt for **Version 1**.

Prompt 1:

"Help me train a PyTorch model for medical imaging. I am working with 3D brain MRI scans in NIfTI (.nii.gz) format to perform binary classification for brain tumor detection (tumor vs. non-tumor)."

Output (Version 1)

Working with 3D brain MRI scans (.nii.gz files) requires volumetric processing rather than standard 2D image pipelines. The most robust framework for this in PyTorch is **MONAI** (Medical Open Network for AI).

1. Key Libraries Needed

```shell
pip install torch monai nibabel
```

2. Complete 3D Brain MRI Binary Classification Pipeline

```python
import os
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
```

```
import monai
from monai.transforms import (
    Compose,
    LoadImaged,
    EnsureChannelFirstd,
    Orientationd,
    Spacingd,
    ScaleIntensityRangePercentilesd,
    ResizeWithPadOrCropd,
    EnsureTyped
)
from monai.data import Dataset
from monai.networks.nets import DenseNet121

# Define MONAI 3D Preprocessing Pipeline for NIfTI Files
train_transforms = Compose([
    LoadImaged(keys=["image"]),
    EnsureChannelFirstd(keys=["image"]),
    Orientationd(keys=["image"], axcodes="RAS"), # Standardize
spatial orientation
    Spacingd(keys=["image"], pixdim=(1.0, 1.0, 1.0),
mode="bilinear"), # Resample to 1mm isotropic voxels
    ScaleIntensityRangePercentilesd(keys=["image"], lower=1,
upper=99, b_min=0.0, b_max=1.0), # Rescale MRI intensities
    ResizeWithPadOrCropd(keys=["image"], spatial_size=(128,
128, 128)), # Standardize volume dimensions
    EnsureTyped(keys=["image"])
])

# Sample File Mapping
data_dicts = [
```

```
    {"image": "data/subject1.nii.gz", "label": 1},
    {"image": "data/subject2.nii.gz", "label": 0},
    # Add all subject file paths here...
]

# Create MONAI Dataset & DataLoader
train_ds = Dataset(data=data_dicts,
transform=train_transforms)
train_loader = DataLoader(train_ds, batch_size=2,
shuffle=True, num_workers=2)

# Set Up Device & 3D Network
device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")

# MONAI provides native 3D convolutional models
(spatial_dims=3)
model = DenseNet121(
    spatial_dims=3,
    in_channels=1,
    out_channels=2
).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

# Training Loop
num_epochs = 10
for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0.0
```

```
    for batch_data in train_loader:
        inputs = batch_data["image"].to(device) # Shape: [B,
1, 128, 128, 128]
        labels = batch_data["label"].to(device) # Shape: [B]

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()

    print(f"Epoch {epoch+1}/{num_epochs}, Average Loss:
{epoch_loss/len(train_loader):.4f}")
```

Assignment Logging: Notes for Version 1

- **What Changed in Prompt:** Added real context specifying the dataset characteristics (3D brain MRI), file format (.nii.gz), and explicit problem type (binary classification for brain tumor detection).
- **What Actually Improved in Output:** The response shifted completely from a basic 2D torchvision.resnet18 toy script to a real 3D volumetric workflow utilizing monai transforms (LoadImaged, Spacingd, RAS reorientation) and a 3D architecture (spatial_dims=3).
- **What Still Failed:** The code assumes a standard fully supervised training setup from scratch, lacks data augmentation, doesn't address class imbalance (common in medical datasets), has no validation/evaluation metrics (like ROC-AUC, sensitivity, or specificity), and doesn't handle low-data regimes or semi-supervised/pretraining setups.

- **What to Try Next:** Select your next single layer from the list (*Clearer Goal, Defined Audience, Specified Output Format, Constraints, Examples, Quality Criteria, Review Instructions, Stated Assumptions, or Verification Requirements*) for **Version 2**.

Prompt 2:

"Help me train a PyTorch model for medical imaging. I am working with 3D brain MRI scans in NIfTI (.nii.gz) format to perform binary classification for brain tumor detection (tumor vs. non-tumor). Instead of standard supervised training from scratch, guide me on setting up a self-supervised pre-training pipeline (like SimCLR or DeepCluster) on unlabeled 3D MRI scans first, followed by fine-tuning on labeled data."

Output (Version 2)

Setting up a self-supervised learning (SSL) pipeline for 3D medical imaging is an effective strategy to leverage unannotated scans. We will use a **SimCLR** (Contrastive Learning) framework adapted for 3D NIfTI volumes using MONAI and PyTorch.

Phase 1: Self-Supervised Pre-Training (SimCLR 3D)

SimCLR creates two augmented views (v_i, v_j) of the same 3D MRI volume and uses a NT-Xent (Normalized Temperature-scaled Cross Entropy) loss to pull positive pairs closer while pushing negative pairs apart.

```python
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from monai.transforms import (
```

```
    Compose, LoadImaged, EnsureChannelFirstd, SpatialPadd,
    RandSpatialCropd, RandFlipd, RandGaussianNoised,
EnsureTyped
)
from monai.data import Dataset
from monai.networks.nets import ViTAutoEnc, DenseNet121

# 1. SSL Data Augmentation: Produces two different views of
the same volume
ssl_transforms = Compose([
    LoadImaged(keys=["image"]),
    EnsureChannelFirstd(keys=["image"]),
    SpatialPadd(keys=["image"], spatial_size=(128, 128, 128)),
    RandSpatialCropd(keys=["image"], roi_size=(96, 96, 96),
random_size=False),
    RandFlipd(keys=["image"], prob=0.5, spatial_axis=0),
    RandGaussianNoised(keys=["image"], prob=0.3, std=0.1),
    EnsureTyped(keys=["image"])
])

class SimCLRDataset(Dataset):
    def __init__(self, data_dicts, transform):
        self.ds = Dataset(data=data_dicts,
transform=transform)
    def __len__(self):
        return len(self.ds)
    def __getitem__(self, idx):
        # Extract two distinct stochastic augmentations of
volume x
        view1 = self.ds[idx]["image"]
        view2 = self.ds[idx]["image"]
```

```
        return view1, view2

# 2. Model with Projection Head
class 3DSimCLRModel(nn.Module):
    def __init__(self, base_encoder, feature_dim=128):
        super().__init__()
        self.encoder = base_encoder # e.g., 3D DenseNet or
ResNet
        # Extract encoder output feature dimension
        num_ftrs = self.encoder.class_head.in_features
        self.encoder.class_head = nn.Identity() # Remove
linear classifier

        # Projection head (MLP)
        self.projector = nn.Sequential(
            nn.Linear(num_ftrs, num_ftrs),
            nn.ReLU(),
            nn.Linear(num_ftrs, feature_dim)
        )

    def forward(self, x):
        features = self.encoder(x)
        projections = self.projector(features)
        return features, projections

# 3. NT-Xent Loss Function
class NTXentLoss(nn.Module):
    def __init__(self, temperature=0.5):
        super().__init__()
        self.temperature = temperature
        self.cosine_sim = nn.CosineSimilarity(dim=-1)
```

```
    def forward(self, z_i, z_j):
        batch_size = z_i.shape[0]
        z = torch.cat([z_i, z_j], dim=0)
        sim_matrix = self.cosine_sim(z.unsqueeze(1),
z.unsqueeze(0)) / self.temperature

        # Labels for positive pairs across concatenated batch
        labels = torch.cat([torch.arange(batch_size, 2 *
batch_size),
                            torch.arange(0,
batch_size)]).to(z.device)

        # Mask out self-contrast
        mask = torch.eye(2 * batch_size,
dtype=torch.bool).to(z.device)
        sim_matrix.masked_fill_(mask, -9e15)

        return nn.CrossEntropyLoss()(sim_matrix, labels)

# Pre-training loop execution
# (Iterate over unannotated NIfTI files for N epochs,
optimization on NTXentLoss)
```

from monai.data import Dataset

Phase 2: Fine-Tuning on Labeled Tumor Data

Once pre-training finishes, freeze or lower the learning rate on the encoder backbone, replace the projection head with a binary classification head, and train on labeled samples.

```python
# Load pre-trained encoder weights
ssl_model =
3DSimCLRModel(base_encoder=DenseNet121(spatial_dims=3,
in_channels=1))
#
ssl_model.load_state_dict(torch.load("pretrained_simclr_3d.pth
"))

# Build Fine-Tuning Model
class BrainTumorClassifier(nn.Module):
    def __init__(self, pretrained_encoder):
        super().__init__()
        self.backbone = pretrained_encoder.encoder
        self.classifier = nn.Linear(1024, 2) # Binary output

    def forward(self, x):
        ft = self.backbone(x)
        return self.classifier(ft)

finetune_model = BrainTumorClassifier(ssl_model).to("cuda")
# Train with standard CrossEntropyLoss and low Learning Rate
(e.g., 1e-5)
```

Assignment Logging: Notes for Version 2

Here are the side-by-side notes for your assignment deliverable:

- **What Changed in Prompt:** Added a clearer goal specifying self-supervised pre-training (SimCLR/DeepCluster) on unlabeled 3D MRI scans prior to downstream fine-tuning, instead of basic supervised training.

- **What Actually Improved in Output:** The output completely transformed its architecture—it introduced a dual-view 3D stochastic augmentation loader, built a custom 3DSimCLRModel with a projection head, implemented a 3D NT-Xent loss function, and explicitly demonstrated how to transfer the pre-trained encoder weights to a downstream binary classifier.
- **What Still Failed:** The code for view generation in SimCLRDataset has a bug (it evaluates self.ds[idx] twice instead of applying two independent random transformations to the *same* raw volume). Additionally, the response lacks concrete validation metrics (e.g., ROC-AUC/Dice), memory optimization constraints (3D SSL requires large effective batch sizes), and structured step-by-step code modularity.
- **What to Try Next:** Select your next single layer from the list (*Defined Audience, Specified Output Format, Constraints, Examples, Quality Criteria, Review Instructions, Stated Assumptions, or Verification Requirements*) for **Version 3**.

Prompt 3:

"Help me train a PyTorch model for medical imaging. I am working with 3D brain MRI scans in NIfTI (.nii.gz) format to perform binary classification for brain tumor detection (tumor vs. non-tumor). Instead of standard supervised training from scratch, guide me on setting up a self-supervised pre-training pipeline (like SimCLR) on unlabeled 3D MRI scans first, followed by fine-tuning on labeled data. Constraints: The execution must fit on a single 16GB GPU (use gradient accumulation or mixed precision torch.cuda.amp), correct the dataset pipeline to ensure two distinct stochastic transformations per volume, and enforce modular code separation between pre-training and fine-tuning."

Output (Version 3)

Project Layout

```
mri_ssl/
├── transforms.py      # Independent 3D spatial/intensity
augmentations
├── dataset.py         # Dual-view SimCLR MONAI Dataset
├── pretrain.py        # Self-supervised SimCLR training
script (AMP + GradAccum)
└── finetune.py        # Downstream binary classifier
fine-tuning script
```

File 1: `transforms.py`

```python
from monai.transforms import (
    Compose, LoadImage, EnsureChannelFirst, SpatialPad,
    RandSpatialCrop, RandFlipd, RandGaussianNoise,
RandAffined, EnsureType
)

# Base loading and spatial preparation
def get_base_transforms():
    return Compose([
        LoadImage(image_only=True),
        EnsureChannelFirst(),
        SpatialPad(spatial_size=(128, 128, 128)),
        EnsureType()
    ])

# Stochastic augmentation pipeline applied independently per
view
```

```
def get_simclr_augmentations():
    return Compose([
        RandSpatialCrop(roi_size=(96, 96, 96),
random_size=False),
        RandFlipd(keys=["img"], prob=0.5, spatial_axis=0),
        RandAffined(keys=["img"], prob=0.5, rotate_range=(0.2,
0.2, 0.2), scale_range=(0.1, 0.1, 0.1)),
        RandGaussianNoise(prob=0.4, mean=0.0, std=0.1),
        EnsureType()
    ])
```

File 2: `dataset.py`

Python

```
from torch.utils.data import Dataset
from transforms import get_base_transforms,
get_simclr_augmentations

class DualViewMRIDataset(Dataset):
    """
    Ensures TWO distinct stochastic augmentations are applied
    to the SAME base volume on every fetch.
    """
    def __init__(self, file_paths):
        self.file_paths = file_paths
        self.base_transform = get_base_transforms()
        self.aug_transform = get_simclr_augmentations()

    def __len__(self):
```

```
        return len(self.file_paths)

    def __getitem__(self, idx):
        # 1. Load base 3D volume once
        raw_vol = self.base_transform(self.file_paths[idx])

        # 2. Apply two independent stochastic passes
        view_1 = self.aug_transform({"img": raw_vol})["img"]
        view_2 = self.aug_transform({"img": raw_vol})["img"]

        return view_1, view_2
```

File 3: `pretrain.py` (SimCLR on 16GB GPU)

Python

```
import torch
import torch.nn as nn
from torch.cuda.amp import autocast, GradScaler
from monai.networks.nets import DenseNet121
from dataset import DualViewMRIDataset
from torch.utils.data import DataLoader

# Encoder + Projection Head
class SimCLR3D(nn.Module):
    def __init__(self):
        super().__init__()
        # Backbone
        self.encoder = DenseNet121(spatial_dims=3,
in_channels=1, out_channels=2)
```

```
        in_features = self.encoder.class_head.in_features
        self.encoder.class_head = nn.Identity()

        # Projection Head
        self.projector = nn.Sequential(
            nn.Linear(in_features, 512),
            nn.ReLU(),
            nn.Linear(512, 128)
        )

    def forward(self, x):
        h = self.encoder(x)
        z = self.projector(h)
        return h, z

def train_simclr(file_paths, epochs=50, batch_size=2,
accum_steps=8):
    device = torch.device("cuda")
    model = SimCLR3D().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
    scaler = GradScaler() # Automatic Mixed Precision (AMP)
for 16GB VRAM

    ds = DualViewMRIDataset(file_paths)
    loader = DataLoader(ds, batch_size=batch_size,
shuffle=True)

    model.train()
    for epoch in range(epochs):
        optimizer.zero_grad()
        for i, (x1, x2) in enumerate(loader):
```

```
            x1, x2 = x1.to(device), x2.to(device)

            with autocast(): # AMP context
                _, z1 = model(x1)
                _, z2 = model(x2)

                # Compute NT-Xent Loss (SimCLR)
                loss = compute_nt_xent_loss(z1, z2) /
accum_steps

            scaler.scale(loss).backward()

            # Gradient Accumulation step to simulate larger
batch size on single GPU
            if (i + 1) % accum_steps == 0 or (i + 1) ==
len(loader):
                scaler.step(optimizer)
                scaler.update()
                optimizer.zero_grad()

    torch.save(model.encoder.state_dict(),
"encoder_pretrained.pth")

def compute_nt_xent_loss(z1, z2, temp=0.1):
    z = torch.cat([z1, z2], dim=0)
    z = nn.functional.normalize(z, dim=1)
    sim_matrix = torch.matmul(z, z.T) / temp

    batch_size = z1.shape[0]
    labels = torch.cat([torch.arange(batch_size, 2 *
batch_size), torch.arange(0, batch_size)]).to(z1.device)
```

```
    mask = torch.eye(2 * batch_size,
dtype=torch.bool).to(z1.device)
    sim_matrix.masked_fill_(mask, -9e15)

    return nn.CrossEntropyLoss()(sim_matrix, labels)
```

File 4: `finetune.py`

Python

```
import torch
import torch.nn as nn
from monai.networks.nets import DenseNet121

class BrainTumorClassifier(nn.Module):
    def __init__(self, pretrained_weights_path=None):
        super().__init__()
        self.encoder = DenseNet121(spatial_dims=3,
in_channels=1, out_channels=2)
        in_features = self.encoder.class_head.in_features
        self.encoder.class_head = nn.Identity()

        if pretrained_weights_path:

self.encoder.load_state_dict(torch.load(pretrained_weights_pat
h))

        self.classifier = nn.Linear(in_features, 2) # Binary:
Tumor vs Non-Tumor
```

```
    def forward(self, x):
        features = self.encoder(x)
        return self.classifier(features)
```

Assignment Logging: Notes for Version 3

Here are the side-by-side notes for your assignment deliverable:

- **What Changed in Prompt:** Added strict engineering constraints: fitting within a 16GB VRAM budget via `torch.cuda.amp` (AMP) and gradient accumulation, fixing the dataset logic to guarantee two distinct transformations per raw volume, and enforcing modular code separation (`transforms.py`, `dataset.py`, `pretrain.py`, `finetune.py`).
- **What Actually Improved in Output:** The output resolved the bug in Version 2 by explicitly creating two separate stochastic augmentations per raw loaded volume inside `DualViewMRIDataset.__getitem__`. It added `autocast()` and `GradScaler()` along with `accum_steps=8` to prevent 3D CUDA Out-Of-Memory (OOM) crashes on 16GB GPUs, and separated the code into explicit production-ready modules.
- **What Still Failed:** The prompt didn't specify evaluation criteria or validation checks—consequently, the code lacks a validation loop, doesn't compute clinical classification metrics (e.g., AUC-ROC, Sensitivity, Specificity), and doesn't handle class imbalance in the labeled fine-tuning stage.
- **What to Try Next:** Select your next single layer from the list (*Defined Audience, Specified Output Format, Examples, Quality Criteria, Review Instructions, Stated Assumptions, or Verification Requirements*) for **Version 4**.

Prompt 4:

"Help me train a PyTorch model for medical imaging. I am working with 3D brain MRI scans in NIfTI (.nii.gz) format to perform binary classification for brain tumor detection (tumor vs. non-tumor). Instead of standard supervised training from scratch, guide me on setting up a self-supervised pre-training pipeline (like SimCLR) on unlabeled 3D MRI scans first, followed by fine-tuning on labeled data. Constraints: The execution must fit on a single 16GB GPU (use gradient accumulation or mixed precision torch.cuda.amp), correct the dataset pipeline to ensure two distinct stochastic transformations per volume, and enforce modular code separation between pre-training and fine-tuning. Quality Criteria: The fine-tuning phase must implement a validation loop that evaluates performance using ROC-AUC, F1-Score, and Sensitivity/Specificity (instead of raw accuracy), and must handle positive class imbalance using weighted loss or sampler."

Output (Version 4)

To meet the quality criteria, we update `finetune.py` with class imbalance mitigation and a validation pipeline calculating clinically meaningful metrics using `scikit-learn`.

Updated Modular File: `finetune.py`

Python

```python
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torch.cuda.amp import autocast, GradScaler
from monai.networks.nets import DenseNet121
import numpy as np
from sklearn.metrics import roc_auc_score, f1_score,
confusion_matrix
```

```
# 1. Downstream Classification Model Definition
class BrainTumorClassifier(nn.Module):
    def __init__(self, pretrained_weights_path=None):
        super().__init__()
        self.encoder = DenseNet121(spatial_dims=3,
in_channels=1, out_channels=2)
        in_features = self.encoder.class_head.in_features
        self.encoder.class_head = nn.Identity()

        if pretrained_weights_path:

self.encoder.load_state_dict(torch.load(pretrained_weights_pat
h))

        self.classifier = nn.Linear(in_features, 2) # Binary:
Non-Tumor (0) vs Tumor (1)

    def forward(self, x):
        features = self.encoder(x)
        return self.classifier(features)

# 2. Validation Function with Clinical Metrics
@torch.no_grad()
def evaluate_validation(model, val_loader, device):
    model.eval()
    all_targets = []
    all_probs = []
    all_preds = []

    for batch in val_loader:
```

```
        images, targets = batch["image"].to(device),
batch["label"].to(device)

        with autocast():
            logits = model(images)
            probs = torch.softmax(logits, dim=1)[:, 1] #
Probability of positive class (Tumor)
            preds = torch.argmax(logits, dim=1)

        all_targets.extend(targets.cpu().numpy())
        all_probs.extend(probs.cpu().numpy())
        all_preds.extend(preds.cpu().numpy())

    all_targets = np.array(all_targets)
    all_probs = np.array(all_probs)
    all_preds = np.array(all_preds)

    # Compute Clinical Metrics
    auc = roc_auc_score(all_targets, all_probs)
    f1 = f1_score(all_targets, all_preds, zero_division=0)

    # Sensitivity (Recall) and Specificity from Confusion
Matrix
    tn, fp, fn, tp = confusion_matrix(all_targets, all_preds,
labels=[0, 1]).ravel()
    sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0

    return {
        "ROC-AUC": auc,
        "F1-Score": f1,
```

```
        "Sensitivity": sensitivity,
        "Specificity": specificity
    }

# 3. Fine-Tuning Execution with Class Imbalance Loss
def run_finetuning(train_loader, val_loader,
pos_weight_val=3.0, epochs=20):
    device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
    model =
BrainTumorClassifier(pretrained_weights_path="encoder_pretrain
ed.pth").to(device)

    # Mitigation for Class Imbalance: Weighted Cross Entropy
    # pos_weight_val = (number of negative samples / number of
positive samples)
    class_weights = torch.tensor([1.0, pos_weight_val],
device=device)
    criterion = nn.CrossEntropyLoss(weight=class_weights)

    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4,
weight_decay=1e-2)
    scaler = GradScaler()

    for epoch in range(epochs):
        model.train()
        train_loss = 0.0

        for batch in train_loader:
            images, targets = batch["image"].to(device),
batch["label"].to(device)
```

```
            optimizer.zero_grad()

            with autocast():
                logits = model(images)
                loss = criterion(logits, targets)

            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

            train_loss += loss.item()

        # Validation Step
        val_metrics = evaluate_validation(model, val_loader,
device)
        print(f"Epoch [{epoch+1}/{epochs}] | Train Loss:
{train_loss/len(train_loader):.4f}")
        print(f"Val AUC: {val_metrics['ROC-AUC']:.4f} | F1:
{val_metrics['F1-Score']:.4f} | "
              f"Sens: {val_metrics['Sensitivity']:.4f} | Spec:
{val_metrics['Specificity']:.4f}")
```

Assignment Logging: Notes for Version 4

Here are the side-by-side notes for your assignment deliverable:

- **What Changed in Prompt:** Added explicit quality criteria requiring a validation evaluation loop computing clinical metrics (ROC-AUC, F1-score, Sensitivity, Specificity) instead of accuracy, alongside handling class imbalance through weighted Cross Entropy Loss (`pos_weight`).
- **What Actually Improved in Output:** The code replaced default accuracy print statements with a dedicated `evaluate_validation()` function that

extracts prediction probabilities, generates a confusion matrix, and computes true positive / true negative rates (Sensitivity/Specificity). It also addressed dataset imbalance directly via `nn.CrossEntropyLoss(weight=class_weights)`.

- **What Still Failed (Honest Moment): This prompt addition made the overall output less usable in practice.** While the fine-tuning script is detailed, focusing heavily on validation metrics bloated `finetune.py` and caused the prompt response to drop the complete `transforms.py`, `dataset.py`, and `pretrain.py` modules from Version 3. A user attempting to run the code end-to-end would now experience broken imports (`transforms`, `dataset`).
- **What to Try Next:** Select your final layer (*Defined Audience, Specified Output Format, Examples, Review Instructions, Stated Assumptions, or Verification Requirements*) for **Version 5** to enforce complete, runnable scripts across all files without missing imports or code truncation.

Prompt 5:

"Help me train a PyTorch model for medical imaging. I am working with 3D brain MRI scans in NIfTI (.nii.gz) format to perform binary classification for brain tumor detection (tumor vs. non-tumor). Instead of standard supervised training from scratch, guide me on setting up a self-supervised pre-training pipeline (like SimCLR) on unlabeled 3D MRI scans first, followed by fine-tuning on labeled data. Constraints: The execution must fit on a single 16GB GPU (use gradient accumulation or mixed precision torch.cuda.amp), correct the dataset pipeline to ensure two distinct stochastic transformations per volume, and enforce modular code separation between pre-training and fine-tuning. Quality Criteria: The fine-tuning phase must implement a validation loop that evaluates performance using ROC-AUC, F1-Score, and Sensitivity/Specificity (instead of raw accuracy), and must handle positive class imbalance using weighted loss or sampler. Specified Output Format: Provide the complete, unabbreviated Python code for all four modules (transforms.py, dataset.py, pretrain.py, finetune.py) in separate markdown blocks. Do not truncate, skip, or use placeholders (e.g., '# previous code here') for any imports or logic."

Output (Version 5)

Module 1: `transforms.py`

Python

```python
import torch
from monai.transforms import (
    Compose,
    LoadImage,
    EnsureChannelFirst,
    SpatialPad,
    EnsureType,
    RandSpatialCrop,
    RandFlipd,
    RandAffined,
    RandGaussianNoised,
    ScaleIntensityRangePercentilesd,
    Orientationd,
    Spacingd
)

def get_base_transforms():
    """
    Standard spatial orientation, resampling, and
normalization for 3D MRI scans.
    """
    return Compose([
        LoadImage(image_only=True),
        EnsureChannelFirst(),
        SpatialPad(spatial_size=(128, 128, 128)),
        EnsureType(dtype=torch.float32)
```

```
    ])

def get_simclr_augmentations():
    """
    Stochastic 3D augmentation pipeline applied independently
to generate views.
    """
    return Compose([
        RandSpatialCrop(roi_size=(96, 96, 96),
random_size=False),
        RandFlipd(keys=["img"], prob=0.5, spatial_axis=0),
        RandAffined(
            keys=["img"],
            prob=0.5,
            rotate_range=(0.2, 0.2, 0.2),
            scale_range=(0.1, 0.1, 0.1),
            mode="bilinear"
        ),
        RandGaussianNoised(keys=["img"], prob=0.4, mean=0.0,
std=0.1),
        EnsureType(dtype=torch.float32)
    ])
```

Module 2: `dataset.py`

Python

```
import torch
from torch.utils.data import Dataset
from transforms import get_base_transforms,
get_simclr_augmentations
```

```
class DualViewMRIDataset(Dataset):
    """
    Dataset wrapper for SimCLR pre-training.
    Loads a single 3D volume and returns TWO distinct
stochastically transformed views.
    """
    def __init__(self, file_paths):
        self.file_paths = file_paths
        self.base_transform = get_base_transforms()
        self.aug_transform = get_simclr_augmentations()

    def __len__(self):
        return len(self.file_paths)

    def __getitem__(self, idx):
        path = self.file_paths[idx]
        raw_vol = self.base_transform(path)

        # Apply two distinct stochastic augmentation passes on
the same volume
        view_1 = self.aug_transform({"img": raw_vol})["img"]
        view_2 = self.aug_transform({"img": raw_vol})["img"]

        return view_1, view_2

class LabeledMRIDataset(Dataset):
    """
    Dataset wrapper for downstream supervised fine-tuning.
    """
    def __init__(self, file_paths, labels):
```

```
        self.file_paths = file_paths
        self.labels = labels
        self.base_transform = get_base_transforms()

    def __len__(self):
        return len(self.file_paths)

    def __getitem__(self, idx):
        path = self.file_paths[idx]
        label = self.labels[idx]
        vol = self.base_transform(path)

        return {
            "image": vol,
            "label": torch.tensor(label, dtype=torch.long)
        }
```

Module 3: `pretrain.py`

Python

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torch.cuda.amp import autocast, GradScaler
from monai.networks.nets import DenseNet121
from dataset import DualViewMRIDataset

class SimCLR3D(nn.Module):
    def __init__(self, feature_dim=128):
```

```
        super().__init__()
        self.encoder = DenseNet121(spatial_dims=3,
in_channels=1, out_channels=2)
        in_features = self.encoder.class_head.in_features
        self.encoder.class_head = nn.Identity()

        self.projector = nn.Sequential(
            nn.Linear(in_features, in_features),
            nn.ReLU(),
            nn.Linear(in_features, feature_dim)
        )

    def forward(self, x):
        features = self.encoder(x)
        projections = self.projector(features)
        return features, projections

def compute_nt_xent_loss(z1, z2, temperature=0.1):
    z1 = F.normalize(z1, dim=1)
    z2 = F.normalize(z2, dim=1)
    z = torch.cat([z1, z2], dim=0)

    sim_matrix = torch.matmul(z, z.T) / temperature
    batch_size = z1.shape[0]

    labels = torch.cat([
        torch.arange(batch_size, 2 * batch_size),
        torch.arange(0, batch_size)
    ]).to(z1.device)
```

```
    mask = torch.eye(2 * batch_size,
dtype=torch.bool).to(z1.device)
    sim_matrix.masked_fill_(mask, -9e15)

    loss_fn = nn.CrossEntropyLoss()
    return loss_fn(sim_matrix, labels)

def train_simclr(unlabeled_file_paths, epochs=50,
batch_size=2, accum_steps=8,
save_path="encoder_pretrained.pth"):
    device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
    model = SimCLR3D().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
    scaler = GradScaler()

    dataset = DualViewMRIDataset(unlabeled_file_paths)
    dataloader = DataLoader(dataset, batch_size=batch_size,
shuffle=True, num_workers=2, pin_memory=True)

    model.train()
    for epoch in range(epochs):
        running_loss = 0.0
        optimizer.zero_grad()

        for step, (x1, x2) in enumerate(dataloader):
            x1, x2 = x1.to(device), x2.to(device)

            with autocast():
                _, z1 = model(x1)
                _, z2 = model(x2)
```

```
                loss = compute_nt_xent_loss(z1, z2) /
accum_steps

            scaler.scale(loss).backward()
            running_loss += loss.item() * accum_steps

            if (step + 1) % accum_steps == 0 or (step + 1) ==
len(dataloader):
                scaler.step(optimizer)
                scaler.update()
                optimizer.zero_grad()

        epoch_loss = running_loss / len(dataloader)
        print(f"Pre-training Epoch [{epoch+1}/{epochs}] |
Loss: {epoch_loss:.4f}")

    torch.save(model.encoder.state_dict(), save_path)
    print(f"Encoder weights saved to {save_path}")

if __name__ == "__main__":
    dummy_unlabeled_paths = [f"data/unlabeled_{i}.nii.gz" for
i in range(10)]
    # train_simclr(dummy_unlabeled_paths)
```

features = self.encoder(x)

Module 4: `finetune.py`

Python

```
import torch
import torch.nn as nn
```

```
from torch.utils.data import DataLoader
from torch.cuda.amp import autocast, GradScaler
from monai.networks.nets import DenseNet121
import numpy as np
from sklearn.metrics import roc_auc_score, f1_score,
confusion_matrix
from dataset import LabeledMRIDataset

class BrainTumorClassifier(nn.Module):
    def __init__(self, pretrained_weights_path=None):
        super().__init__()
        self.encoder = DenseNet121(spatial_dims=3,
in_channels=1, out_channels=2)
        in_features = self.encoder.class_head.in_features
        self.encoder.class_head = nn.Identity()

        if pretrained_weights_path:

self.encoder.load_state_dict(torch.load(pretrained_weights_pat
h))
            print(f"Loaded pre-trained encoder weights from
{pretrained_weights_path}")

        self.classifier = nn.Linear(in_features, 2)

    def forward(self, x):
        features = self.encoder(x)
        return self.classifier(features)

@torch.no_grad()
def evaluate_validation(model, val_loader, device):
```

```
    model.eval()
    all_targets = []
    all_probs = []
    all_preds = []

    for batch in val_loader:
        images = batch["image"].to(device)
        targets = batch["label"].to(device)

        with autocast():
            logits = model(images)
            probs = torch.softmax(logits, dim=1)[:, 1]
            preds = torch.argmax(logits, dim=1)

        all_targets.extend(targets.cpu().numpy())
        all_probs.extend(probs.cpu().numpy())
        all_preds.extend(preds.cpu().numpy())

    all_targets = np.array(all_targets)
    all_probs = np.array(all_probs)
    all_preds = np.array(all_preds)

    auc = roc_auc_score(all_targets, all_probs) if
len(np.unique(all_targets)) > 1 else 0.5
    f1 = f1_score(all_targets, all_preds, zero_division=0)

    cm = confusion_matrix(all_targets, all_preds, labels=[0,
1])
    if cm.shape == (2, 2):
        tn, fp, fn, tp = cm.ravel()
        sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0.0
```

```
        specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
    else:
        sensitivity, specificity = 0.0, 0.0

    return {
        "ROC-AUC": auc,
        "F1-Score": f1,
        "Sensitivity": sensitivity,
        "Specificity": specificity
    }

def train_downstream(
    train_paths,
    train_labels,
    val_paths,
    val_labels,
    pretrained_weights_path="encoder_pretrained.pth",
    epochs=20,
    batch_size=2
):
    device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
    model =
BrainTumorClassifier(pretrained_weights_path=pretrained_weight
s_path).to(device)

    num_neg = sum(1 for l in train_labels if l == 0)
    num_pos = sum(1 for l in train_labels if l == 1)
    pos_weight_val = num_neg / max(num_pos, 1)
```

```
    class_weights = torch.tensor([1.0, pos_weight_val],
dtype=torch.float32, device=device)
    criterion = nn.CrossEntropyLoss(weight=class_weights)

    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4,
weight_decay=1e-2)
    scaler = GradScaler()

    train_ds = LabeledMRIDataset(train_paths, train_labels)
    val_ds = LabeledMRIDataset(val_paths, val_labels)

    train_loader = DataLoader(train_ds, batch_size=batch_size,
shuffle=True, num_workers=2)
    val_loader = DataLoader(val_ds, batch_size=batch_size,
shuffle=False, num_workers=2)

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0

        for batch in train_loader:
            images = batch["image"].to(device)
            targets = batch["label"].to(device)

            optimizer.zero_grad()
            with autocast():
                logits = model(images)
                loss = criterion(logits, targets)

            scaler.scale(loss).backward()
            scaler.step(optimizer)
```

```
            scaler.update()

            running_loss += loss.item()

        val_metrics = evaluate_validation(model, val_loader,
device)
        print(f"Epoch [{epoch+1}/{epochs}] | Train Loss:
{running_loss/len(train_loader):.4f}")
        print(f"Val AUC: {val_metrics['ROC-AUC']:.4f} | F1:
{val_metrics['F1-Score']:.4f} | "
              f"Sens: {val_metrics['Sensitivity']:.4f} | Spec:
{val_metrics['Specificity']:.4f}")

if __name__ == "__main__":
    dummy_train_paths = [f"data/train_{i}.nii.gz" for i in
range(10)]
    dummy_train_labels = [0, 1, 0, 1, 0, 0, 1, 0, 1, 1]
    dummy_val_paths = [f"data/val_{i}.nii.gz" for i in
range(4)]
    dummy_val_labels = [0, 1, 0, 1]
    # train_downstream(dummy_train_paths, dummy_train_labels,
dummy_val_paths, dummy_val_labels)
```

Assignment Logging: Notes for Version 5

Here are the side-by-side notes for your assignment deliverable:

- **What Changed in Prompt:** Added a specified output format layer requiring complete, unabbreviated code blocks for all 4 modules (`transforms.py`, `dataset.py`, `pretrain.py`, `finetune.py`) without truncations or placeholders.

- **What Actually Improved in Output:** Reinstated all missing pre-training, augmentation, and dataset loader files from Version 3 alongside the validation/class-imbalance logic from Version 4. Added missing helper classes (`LabeledMRIDataset`), safety checks for single-class validation splits in `evaluate_validation()`, and executable `__main__` entry points.
- **What Still Failed:** The prompt lacks explicit review/testing instructions to automatically handle missing local `.nii.gz` synthetic file creation during execution testing (running as-is requires valid local NIfTI files on disk).
- **What to Try Next:** The 5-run prompt ladder is now complete. We can now assemble the final clean, reusable prompt document!

Final Deliverable: Clean Reusable Prompt 👍:

Act as a Senior AI Medical Imaging Engineer. Provide a complete, production-ready PyTorch implementation for 3D Brain MRI tumor detection using Self-Supervised Learning (SimCLR pre-training) followed by downstream fine-tuning.

Task Requirements:
1. Context & Dataset: 3D brain MRI scans stored in NIfTI (.nii.gz) format performing binary classification (0: Non-Tumor, 1: Tumor).
2. Pipeline Strategy:
  - Pre-training phase: 3D SimCLR contrastive learning on unlabeled NIfTI scans.
  - Downstream phase: Transfer pre-trained encoder weights to a binary classification model fine-tuned on labeled scans.
3. Hardware Constraints: Single 16GB GPU VRAM limit. Must use PyTorch AMP (torch.cuda.amp) and gradient accumulation (e.g., accum_steps=8).
4. Dual Augmentation Pipeline: In dataset.py, ensure two distinct stochastic transformations are generated per volume for SimCLR contrastive loss.
5. Quality Criteria:
  - Handle class imbalance in downstream fine-tuning using class-weighted Cross Entropy Loss.

- Compute validation metrics: ROC-AUC, F1-Score, Sensitivity (Recall), and Specificity.
6. Formatting Constraint: Provide complete, fully unabbreviated Python code across 4 separate modular files (transforms.py, dataset.py, pretrain.py, finetune.py). Do not truncate code or use placeholders. Include main execution blocks.